

Symbols And Nomenclature

Intro And Scope

This document defines the notation used throughout the theory library and, later, in the software implementation notes. Its purpose is to remove ambiguity before entering derivations, algorithms, and design decisions.

The project works with polynomial trajectories in 1D first and then extends the same concepts to 3D when needed. The 2D case is intentionally omitted because it is considered a direct conceptual subset of the 3D case.

Orientation is included here only as reserved notation. Full theoretical treatment of orientation is explicitly postponed to a later phase.

Guiding Conventions

- Time-dependent quantities are written as functions of time, for example $x(t)$.
- Scalar quantities are written in plain italic notation, for example $x(t)$, $v(t)$, T .
- Three-dimensional vectors are written in bold, for example $\mathbf{p}(t)$.
- Initial and final values use subscripts 0 and f , for example x_0 , x_f , $\mathbf{p}_0$, $\mathbf{p}_f$.
- Segment indices use subscripts k , for example $x_k(t)$.
- Normalized time is written as τ , while absolute time is written as t .

Time Variables

The following symbols are used for time:

- t : absolute time variable.
- t_0 : initial time.
- t_f : final time.
- $T = t_f - t_0$: motion duration.
- τ : normalized time variable, usually defined as

$$\tau = \frac{t - t_0}{T}, \quad \tau \in [0, 1].$$

This distinction is useful because some derivations are clearer in absolute time, while others are more compact in normalized time.

Position And Trajectory In 1D

In one dimension, the trajectory is represented by a scalar position law:

$$x(t)$$

For a polynomial of degree n , the canonical form is:

$$x(t) = \sum_{i=0}^n a_i t^i$$

where:

- a_i are the polynomial coefficients;
- n is the polynomial degree.

When the project specifically refers to the ninth-degree polynomial, the notation becomes:

$$x(t) = a_0 + a_1t + a_2t^2 + \dots + a_9t^9$$

The corresponding Horner form is written as:

$$x(t) = (((((((((a_9t + a_8)t + a_7)t + a_6)t + a_5)t + a_4)t + a_3)t + a_2)t + a_1)t + a_0)$$

This document only fixes the notation. The analytical comparison between canonical and Horner form belongs to `NinthDegreePolynomialTheory`.

Derivatives In 1D

The project adopts the following derivative chain:

- position: $x(t)$
- velocity: $\dot{x}(t)$
- acceleration: $\ddot{x}(t)$
- jerk: $x^{(3)}(t)$
- snap: $x^{(4)}(t)$
- crackle: $x^{(5)}(t)$
- pop: $x^{(6)}(t)$

For quick reading, the shorthand notation below is also accepted when useful:

$$v(t) = \dot{x}(t), \quad a(t) = \ddot{x}(t), \quad j(t) = x^{(3)}(t), \quad s(t) = x^{(4)}(t)$$

where:

- $v(t)$ denotes velocity;
- $a(t)$ denotes acceleration;
- $j(t)$ denotes jerk;
- $s(t)$ denotes snap.

For higher derivatives beyond snap, the explicit derivative notation $x^{(5)}(t)$ and $x^{(6)}(t)$ is preferred because it is less ambiguous than introducing too many extra letters.

Boundary Conditions In 1D

A boundary condition is a value imposed at the start or end of a segment. The standard notation is:

$$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f, s_0, s_f$$

These symbols mean:

- x_0, x_f : initial and final position;
- v_0, v_f : initial and final velocity;
- a_0, a_f : initial and final acceleration;
- j_0, j_f : initial and final jerk;
- s_0, s_f : initial and final snap.

Not every polynomial degree requires the same set of boundary conditions. In later documents, boundary conditions will always be classified as:

- necessary;
- possible;
- redundant.

This classification depends on the polynomial degree under study.

State Notation

When it is useful to describe the motion condition at a single endpoint, the project uses the concept of state.

In 1D, a state can be written as:

$$\mathcal{S}_{1D} = (x, v, a, j, s)$$

or, if endpoint notation is needed:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

The symbol $\mathcal{S}$ (calligraphic) is used here as a compact state container. It is deliberately distinct, both visually and semantically, from s / $\mathbf{s}$, which remain reserved exclusively for snap (see *Derivatives In 1D*) – no disambiguation is required when both appear in the same context, since the two symbols no longer collide.

Position And Trajectory In 3D

In three dimensions, position is represented by a vector:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

The three scalar component laws are treated with the same notation already defined for 1D.

This means that a 3D polynomial trajectory can be understood as three coordinated scalar polynomial laws:

$$x(t), \quad y(t), \quad z(t)$$

or, compactly:

$$\mathbf{p}(t) = [x(t), y(t), z(t)]^T$$

Derivatives In 3D

The derivative hierarchy extends component-wise:

$$\dot{\mathbf{p}}(t), \quad \ddot{\mathbf{p}}(t), \quad \mathbf{p}^{(3)}(t), \quad \mathbf{p}^{(4)}(t)$$

The compact names are:

- velocity vector: $\mathbf{v}(t) = \dot{\mathbf{p}}(t)$
- acceleration vector: $\mathbf{a}(t) = \ddot{\mathbf{p}}(t)$
- jerk vector: $\mathbf{j}(t) = \mathbf{p}^{(3)}(t)$
- snap vector: $\mathbf{q}(t) = \mathbf{p}^{(4)}(t)$

The symbol $\mathbf{q}(t)$ is reserved here for vector snap only to avoid overloading the scalar letter s . If this creates confusion with quaternion notation in a future document, the explicit form $\mathbf{p}^{(4)}(t)$ should be preferred.

Boundary Conditions In 3D

The endpoint notation extends directly from the 1D case:

$$\mathbf{p}_0, \mathbf{p}_f, \mathbf{v}_0, \mathbf{v}_f, \mathbf{a}_0, \mathbf{a}_f, \mathbf{j}_0, \mathbf{j}_f, \mathbf{p}_0^{(4)}, \mathbf{p}_f^{(4)}$$

Each vector quantity contains three scalar components. For example:

$$\mathbf{v}_0 = \begin{bmatrix} v_{x,0} \\ v_{y,0} \\ v_{z,0} \end{bmatrix}$$

This reinforces the main project idea: 3D treatment is built from the 1D theory, then extended by vector composition.

Segments, Multi-Segment Trajectories, And Blending

For a single segment, the trajectory law may be written as:

$$x_k(t)$$

in 1D, or:

$$\mathbf{p}_k(t)$$

in 3D, where k is the segment index.

A complete multi-segment trajectory is represented as an ordered set:

$$\mathcal{T} = \{\text{segment}_1, \text{segment}_2, \dots, \text{segment}_N\}$$

When blending is introduced between consecutive segments, the project uses the following notation:

- $b_{k \rightarrow k+1}^{start}$: point where blending toward the next segment starts;
- $b_{k \rightarrow k+1}^{end}$: point where blending from the previous segment ends.

If the context is explicitly geometric, those symbols may be written as positions:

$$x_{b,k}^{start}, \quad x_{b,k}^{end}$$

in 1D, or:

$$\mathbf{p}_{b,k}^{start}, \quad \mathbf{p}_{b,k}^{end}$$

in 3D.

Constraint Notation

The project uses generic constraint symbols first, then specific ones when needed.

Generic notation:

$$C = \{\text{type}, \text{value}\}$$

Typical scalar limits in 1D are:

- v_{max} : maximum admissible velocity;
- a_{max} : maximum admissible acceleration;
- j_{max} : maximum admissible jerk;
- s_{max} : maximum admissible snap.

When a constant-velocity sub-segment is required, the notation is:

$$x_{cv}^{start}, \quad x_{cv}^{end}, \quad v_{cv}$$

in 1D, and:

$$\mathbf{p}_{cv}^{start}, \quad \mathbf{p}_{cv}^{end}, \quad v_{cv}$$

in 3D.

The scalar v_{cv} denotes the requested constant speed magnitude. If direction matters in 3D, the corresponding constant velocity vector should be written as:

$$\mathbf{v}_{cv}$$

Reserved Orientation Notation

Orientation is not developed here, but the following symbols are reserved so that future documents remain coherent:

- Euler angles: ϕ, θ, ψ
- rotation matrix: $\mathbf{R}$
- quaternion: $\mathbf{q}$ or q , depending on the chosen convention

Because $\mathbf{q}$ may also be used for snap in some literature, future orientation documents must explicitly declare the chosen convention before use. Until then:

- use $\mathbf{p}^{(4)}(t)$ when referring to vector snap in potentially ambiguous contexts;
- use $\mathbf{R}$, ϕ , θ , ψ for reserved orientation notation;
- avoid quaternion derivations for now.

Procedure For Later Documents

To keep the full library consistent, every later document should follow this practical sequence:

1. start from the notation defined here;
2. introduce only the symbols actually needed by that topic;
3. keep the 1D derivation as the base case;
4. extend to 3D by component-wise interpretation or vector form;
5. declare any new symbol locally if it is not already defined here.

This simple procedure is important for study: it avoids re-learning a different notation in every document.

Worked Symbolic Considerations

The notation can already be read in a symbolic way without numerical data.

Example 1, 1D endpoint problem:

$$x(t) : (x_0, v_0, a_0, j_0, s_0) \rightarrow (x_f, v_f, a_f, j_f, s_f)$$

Example 2, 3D endpoint problem:

$$\mathbf{p}(t) : (\mathbf{p}_0, \mathbf{v}_0, \mathbf{a}_0) \rightarrow (\mathbf{p}_f, \mathbf{v}_f, \mathbf{a}_f)$$

Example 3, normalized-time formulation:

$$x(\tau) = \sum_{i=0}^n \alpha_i \tau^i, \quad \tau \in [0, 1]$$

where α_i are the coefficients expressed in normalized time.

These examples are intentionally symbolic because the project prioritizes structural understanding before numerical substitution.

Open Questions

- For each polynomial degree, what is the minimal boundary-condition set that determines the coefficients uniquely?
- When discussing 3D constraints, should limits be imposed component-wise or on vector magnitude?
- In future orientation work, should quaternion notation use q or $\mathbf{q}$ once snap notation is already present?

Related Documents

- `06-theory-library-roadmap.md`
- `NinthDegreePolynomialTheory`
- `BoundaryConditions`
- `PolynomialCoefficientDetermination`